semicolon to denote the completion of a command, the following code will not compile successfully and will generate an error message:

```
package main
import "fmt"

func main()
{
     fmt.Println("Hello, world!")
}
```

I called the Go file *hello2.go* and I am trying to compile it as usual:

```
$ go run hello2.go
# command-line-arguments
./hello2.go:5: syntax error: unexpected semicolon or newline
before {
```

The formal explanation for this error is that Go requires the use of semicolons as statement terminators in many contexts, and the compiler is able to automatically insert the required semicolons at the end of non-blank lines. Putting the brace ({) on its own line will put a semicolon at the end of the previous line, which obviously generates the error and the presented error message. The good thing is that you no longer need to worry about brace *placement* as in other programming languages.

What is Go?

Go is a relatively new programming language that tries to be modern, efficient and pleasant to the programmer. The *main* characteristics of Go are:

- It is fast, fun and productive. After all, programming must be fun and Go tries to make the programming experience entertaining!
- Go code tries to be both *clean* and *simple*.
- It supports procedural programming.
- It supports object-oriented programming.
- It supports concurrent programming.
- It supports distributed programming.
- It supports garbage-collection.
- Go programs are written in plain text Unicode format using UTF-8 encoding.
- It aims for high-speed compilation.
- It is designed to scale efficiently.
- Go aims to be type safe and memory safe.
- It can be used to build Web applications. It also supports Google's App Engine.
- It is a systems language, in the sense that it is expected to write software such as Web servers.
- The syntax of Go looks like that of a C-based language.
- Go does not have a *preprocessor*.

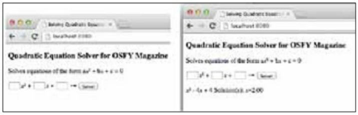

Figure 1: Using the *quad* program from your browser

- The grammar of Go is regular and simple: a few keywords that can be parsed without a symbol table.
- Go keeps concepts *orthogonal* because a few orthogonal features work better than many overlapping ones.
- It can replace both C++ and Java. Personally, I do not like programming in Java but I used to like C++, although, nowadays, I rarely write in C++ because I believe that it has become a big and complicated programming language.
- A consequence of Go being fast to compile is that it can be also used in situations where scripting languages are used.

I think that this is enough for theoretical information. It is time to see some real and working Go code!

A simple Go example

The following Go program (file *countLines.go*) reads a text file and counts the number of lines it has: